

Playbook — Tech Leadership & Team Building

Guida operativa interna — uso personale

Questo documento è il mio manuale di lavoro. Non lo legge il cliente. Qui scrivo quello che so davvero fare, come lo faccio, con quale mindset. Il tono è da esperto a se stesso: diretto, senza fronzoli.

SEZIONE 1 — Il Servizio: Posizionamento

1.1 Cosa vendo

Tre prodotti, tre contesti di applicazione diversi.

Team Assessment — fotografia dello stato attuale del team.

Durata: 1-2 giorni on-site o remoti. Output: report con Engineering Maturity Score per dimensione, 3 priorità di intervento, raccomandazioni operative. Prezzo: €8.000 flat.

Leadership Program — trasformazione della cultura tecnica nel tempo.

Durata: 3 mesi, strutturati per milestone. Include assessment iniziale, sessioni settimanali/bisettimanali, coaching diretto all'Engineering Manager, revisione dei processi. Prezzo: €5.000–€8.000/mese (dipende da numero di persone nel team e intensità del lavoro). Rinnovo mensile dopo il primo trimestre.

Workshop Engineering Excellence — formazione intensiva su un tema specifico (feedback culture, 1-on-1 efficaci, psychological safety, decision making, hiring tecnico). Durata: 1 giorno. Prezzo: €5.000–€8.000/giorno in base al numero di partecipanti e alla personalizzazione richiesta.

1.2 Cliente ideale

Contesto aziendale: team tech 5–30 persone. Startup Series A/B o scale-up che ha appena assunto il primo Engineering Manager. Azienda tradizionale che sta costruendo un team tech interno.

Il problema reale che hanno (non quello che descrivono):

- L'Engineering Manager è stato promosso dal basso perché era il migliore sviluppatore. Non ha mai ricevuto formazione sulla leadership. Ora gestisce persone senza strumenti. Soffre in silenzio.
- Il CTO vede il team che non scala, i deadline non rispettati, le persone che se ne vanno. Vuole "fare qualcosa" ma non sa cosa.
- Il team ha buone persone ma non collabora bene. I senior non condividono la conoscenza. I junior si sentono abbandonati. Le retrospective non esistono o sono rituali vuoti.

Il CEO/CTO che ingaggerà il servizio ha queste caratteristiche:

- Sa che c'è un problema di leadership, non solo di processo.
 - Ha già tentato qualcosa da solo (un corso qui, un libro là) senza risultati duraturi.
 - È disposto a investire tempo e denaro su una trasformazione reale.
 - Non cerca magia — cerca metodo.
-

1.3 Red flag — cliente che non ingaggio (o non rinnovo)

"Vogliamo motivare il team" senza nessuna disponibilità a cambiare i processi o la struttura organizzativa. La motivazione non si installa come plugin. Se il sistema premia i comportamenti sbagliati, nessuna formazione regge.

"Abbiamo bisogno di un training box da spuntare" — HR che vuole giustificare il budget formazione. L'engagement sarà superficiale, i risultati non saranno misurati, il lavoro non lascerà traccia.

CTO assente durante il programma. Se la leadership senior non è ingaggiata e non cambia il proprio comportamento, il lavoro sull'Engineering Manager è inutile — o peggio, crea attrito.

Aspettativa di risultati in 2 settimane. I comportamenti cambiano lentamente. Un mese per stabilizzare un nuovo comportamento è il minimo. Chi vuole risultati in 2 settimane vuole optics, non cambiamento.

1.4 Come mi differenzio

Non sono un executive coach generico con qualche nozione di "agile". Vengo dal tech — ho costruito sistemi, ho gestito team fino a 20 persone, ho vissuto le stesse crisi che vive chi mi ingaggia.

Quello che porto in più rispetto a un consulente organizzativo tradizionale:

- Credibilità tecnica: posso parlare con i developer senza dover fingere. Capisco il debito tecnico, le pressioni dello sprint, il dolore di un sistema legacy. I developer lo sentono subito — abbassano le difese.
- NLP Counselor certificato (PNL Meta): non uso il coaching come strumento generico. Uso tecniche precise per calibrare, riformulare, creare rapport e sbloccare comportamenti. Questo non è visibile al cliente, ma cambia radicalmente la qualità delle conversazioni.
- Yoga Teacher: porto una comprensione del corpo, del respiro, dell'attenzione. Non la vendo come competenza ma la uso — soprattutto per gestire conversazioni ad alta tensione emotiva senza perdere il centro.
- Risultati misurati: +50% team satisfaction, +30% development velocity in contesto TicketOne/Eventim. Questi numeri hanno storia, metodo, contesto. Posso raccontarli.

SEZIONE 2 — Il Framework: Engineering Excellence

2.1 Le 4 dimensioni

Ogni team tech può essere valutato su quattro assi. Non sono indipendenti — si influenzano a vicenda. Ma sono separabili per l'analisi.

Dimensione 1 — Technical Quality

Cosa misura: la qualità del lavoro tecnico prodotto dal team.

Indicatori osservabili:

- Deployment frequency (quante volte si deploia in produzione per settimana)
- Lead time for changes (da commit a produzione)
- Change failure rate (% di deploy che causano incidenti)
- MTTR — Mean Time to Restore (quanto si impiega a ripristinare dopo un incidente)

- Test coverage (% codebase coperta da test automatici)
- Numero di critical/high bug in produzione negli ultimi 30 giorni
- Tempo medio per chiudere un issue tecnico aperto da >30 giorni

Come misurarli: DORA metrics via CI/CD (GitLab, GitHub Actions). Test coverage dai report CI. Bug count da JIRA/Linear. Lead time calcolabile da git log.

Benchmark [probabile] per team 5–15 persone, scala-up:

- Deployment frequency: >1/settimana (livello minimo accettabile) — >1/giorno (livello ottimale)
- Lead time: <1 settimana (accettabile) — <1 giorno (ottimale)
- Change failure rate: <15% (accettabile) — <5% (ottimale)
- MTTR: <1 giorno (accettabile) — <1 ora (ottimale)

Dimensione 2 — Team Health

Cosa misura: la qualità delle relazioni, la sicurezza psicologica, la capacità del team di funzionare come sistema.

Indicatori osservabili:

- Turnover ultimi 12 mesi (% di persone uscite)
- Numero di conversazioni difficili gestite (feedback critico dato, conflitti affrontati)
- Partecipazione attiva nelle retrospective (non solo presenza)
- Segnali di burnout: ore extra sistematiche, assenze, qualità del codice in calo
- Psychological safety: le persone ammettono errori? Fanno domande "stupide"? Dissentono in riunione?

Come misurarli: survey anonima (template in Sezione 7). 1-on-1 con osservazione diretta. Proxy: frequenza delle interruzioni ferie, numero di giorni di malattia, tono nelle code review.

Benchmark [probabile]:

- Turnover annuo: <15% (accettabile) — <8% (ottimale per tech)
- Participation rate in retro: >70% (accettabile) — >90% (ottimale)
- Survey psychological safety score: >3.5/5 (accettabile) — >4.2/5 (ottimale)

Dimensione 3 — Delivery Effectiveness

Cosa misura: la capacità del team di consegnare valore in modo prevedibile ed efficiente.

Indicatori osservabili:

- Sprint commitment accuracy (% di story points consegnati vs pianificati)
- Waste rate (tempo speso in lavoro che non produce valore: riunioni inutili, rework, blocchi)
- WIP medio (work in progress simultaneo per persona)
- Backlog health: % di ticket con criteri di accettazione chiari

Come misurarli: JIRA/Linear report. Sprint velocity history. Osservazione diretta delle cerimonie.

Benchmark [probabile]:

- Sprint commitment accuracy: >70% (accettabile) — >85% (ottimale)
- WIP per persona: <3 task attivi (accettabile) — <2 (ottimale)

Dimensione 4 — Leadership Capability

Cosa misura: la qualità della leadership esercitata all'interno del team.

Indicatori osservabili:

- 1-on-1 regolari con ogni membro del team (cadenza, qualità, follow-up)
- Feedback culture: il feedback critico viene dato? Viene ricevuto bene?
- Decision ownership: le decisioni vengono prese al livello giusto, non sempre escalate?
- Growth mindset: le persone vengono aiutate a crescere (skill, carriera, autonomia)?
- Delega reale: l'EM fa ancora lavoro tecnico invece di delegare?

Come misurarli: survey 1-on-1 quality. Osservazione in sessione. Interview con il team.

Benchmark [probabile]:

- 1-on-1 cadenza: almeno ogni 2 settimane (accettabile) — settimanale (ottimale)
- Feedback bidirezionale: >80% del team riceve e dà feedback regolare (ottimale)

2.2 Engineering Maturity Model

Cinque livelli per ogni dimensione. Il livello non è un giudizio — è una mappa per capire dove si è e dove si può andare.

Livello	Nome	Descrizione
1	Chaos	Reattivo. Nessun processo. Le decisioni vengono prese in emergenza. I problemi vengono "risolti" senza capire le cause.
2	Managed	Processi base esistono ma non seguiti in modo consistente. Dipende molto dalle singole persone. Se una persona chiave se ne va, il p
3	Defined	Processi definiti, documentati, seguiti dal team. Prevedibilità media. Il nuovo membro del team capisce come funziona senza dipender
4	Quantified	Metriche e dati guidano le decisioni. Il team sa dove sta andando bene e dove no. Le retrospective producono azioni misurabili.
5	Optimizing	Cultura del miglioramento continuo. Il team si auto-corregge. La leadership è distribuita. I problemi vengono anticipati, non sol

Come usare il modello: per ogni dimensione, assegno un livello basandomi sull'assessment. L'obiettivo non è portare tutto a livello 5 — è portare le dimensioni critiche al livello giusto per il contesto. Un team di 5 persone in una startup può funzionare benissimo a livello 3 su delivery e livello 4 su team health.

SEZIONE 3 — Il Team Assessment (1-2 giorni)

3.1 Preparazione (prima di arrivare)

Dati da richiedere al cliente con almeno una settimana di anticipo:

Metriche operative (se disponibili):

- Deployment frequency ultimi 3 mesi
- MTTR ultimi 3 incidenti
- Lead time medio per feature (da commit a produzione)
- Turnover ultimi 12 mesi (numero di uscite e ingressi)
- Satisfaction score se esiste (NPS interno, survey annuale)

Se non hanno questi dati, è già un dato: siamo a livello 1-2 su delivery effectiveness.

Survey anonima pre-assessment: da mandare almeno 48h prima al team. Template in Sezione 7. Dà una baseline quantitativa prima delle interview e permette di identificare aree di attenzione.

Organigramma del team: chi riporta a chi, da quanto tempo, quale seniority. Da integrare con la lista dei partecipanti alle interview.

Calendario: bloccare i 30 minuti per ogni 1-on-1 nel giorno 1. Massimo 8 interviste in un giorno — oltre non si ascolta più bene.

3.2 Giorno 1 — Interview

Ordine delle interview: iniziare dagli IC (individual contributors), finire con l'Engineering Manager. Questo mi dà il quadro dal basso prima di ascoltare la versione del leader. Evito di essere influenzato dalla narrativa manageriale.

Setup fisico o digitale: camera accesa se in video call. Niente note visibili sullo schermo (scrive su carta o su documento separato). Tono: curiosità genuina, nessun giudizio visibile.

Apertura standard: "Questo è uno spazio confidenziale. Quello che mi dici non viene citato direttamente nel report con nome e cognome. Uso le informazioni in modo aggregato. L'obiettivo non è trovare colpevoli — è capire come funziona il sistema e dove posso aiutarti."

Domande per Developer (IC)

Prima parte — situazione attuale:

- "Descrivi come lavori in una settimana tipo. Cosa ti dà più soddisfazione? Cosa ti drena più energia?"
- "Quando hai un blocco tecnico, come lo risolvi? A chi vai?"
- "Quante volte a settimana devi aspettare qualcuno o qualcosa prima di poter andare avanti?"

Seconda parte — collaborazione e feedback:

- "Come ti senti riguardo alla qualità del codice che produce il team? C'è qualcosa che ti fa arrabbiare?"
- "Quando il tuo codice va in code review, com'è l'esperienza? Ti aspetti feedback utile?"
- "Hai mai detto al tuo manager qualcosa che non funzionava? Com'è andata?"

Terza parte — prospettiva personale:

- "Se potessi cambiare una cosa nel modo in cui lavora questo team, cosa cambieresti?"
- "Ti senti supportato nella tua crescita professionale qui?"
- "Cosa ti farebbe pensare a trovare un altro lavoro?"

Nota NLP: la domanda "cosa ti farebbe pensare a trovare un altro lavoro" è una domanda outcome-frame negativa. Di solito sblocca informazioni che le persone non dicono direttamente (retribuzione, mancanza di riconoscimento, relazioni difficili). Ascoltare la risposta senza reagire. La non-risposta o la risposta evasiva è informativa quanto la risposta diretta.

Domande per Tech Lead

Prima parte — responsabilità e decision making:

- "Come descrivi il tuo ruolo nel team? Cosa fai che un senior developer non fa?"
- "Quando c'è una decisione tecnica importante, come viene presa? Chi ha l'ultima parola?"
- "Hai mai dovuto dire no a una scelta tecnica proposta dall'alto? Com'è andata?"

Seconda parte — relazioni e influenza:

- "Come gestisci i disaccordi tecnici nel team?"
- "C'è qualcuno nel team con cui è difficile lavorare? Come lo gestisci?"
- "L'Engineering Manager ti supporta nelle decisioni che prendi? O preferisce prendere lui?"

Terza parte — crescita e limiti:

- "Dove ti senti bloccato nel tuo ruolo di leader tecnico?"
- "Cosa non sai fare che ti piacerebbe imparare?"

Domande per Engineering Manager

Prima parte — auto-percezione del ruolo:

- "Come descriveresti il tuo stile di management a qualcuno che non ti conosce?"
- "Quali sono i tuoi 3 KPI di successo come EM? Come li misuri?"
- "Quanto tempo passi su lavoro tecnico diretto vs lavoro di management?"

Seconda parte — energia e sostenibilità:

- "Cosa ti dà energia in questo ruolo? Cosa te la toglie?"
- "C'è qualcosa che stai rimandando perché non sai come affrontarlo?"
- "Come stai, davvero? Scala da 1 a 10 — dove sei questa settimana?"

Terza parte — il team dalla sua prospettiva:

- "Chi nel team ti preoccupa? Perché?"
- "Chi nel team ti sorprende positivamente?"
- "Se potessi cambiare una cosa nel funzionamento del team domani mattina, cosa sarebbe?"

Nota NLP — calibrazione durante le interview: osservare i segnali di incongruenza (voce che si abbassa quando si parla di un argomento specifico, risposta eccessivamente rapida su certi temi, risate nervose). Questi segnali indicano dove c'è tensione non detta. Non insistere sul momento — tornare sull'argomento verso la fine della conversazione con una domanda aperta leggera: "Tornando a quello che hai detto su X — c'è altro che vuoi aggiungere?"

Domande per Product Manager (se presente)

- "Come descriveresti la relazione tra tech e prodotto in questo team? È una collaborazione o è transazionale?"
- "Quando una feature viene consegnata in ritardo, qual è la dinamica che si crea?"
- "Ti senti ascoltato dal team tecnico? E tu, ascolti abbastanza il team tecnico sulle vincoli tecnici?"
- "Cosa cambieresti nel processo di discovery/delivery?"

3.3 Giorno 2 — Synthesis

Costruire il Team Health Score

Tabella di sintesi: per ogni dimensione del framework (Technical Quality, Team Health, Delivery Effectiveness, Leadership Capability), assegno un livello di maturità (1–5) basandomi su:

- Dati quantitativi (survey, metriche)
- Dati qualitativi (interview)
- Osservazione diretta (se ho partecipato a qualche cerimonia)

Ogni punteggio va accompagnato da 2–3 evidenze specifiche (anonime) che lo supportano.

Identificare i 3 interventi prioritari

Non tutti i problemi sono uguali. Il criterio di priorità è:

- Impatto sul business a breve termine (rischio perdita persone chiave, rischio delivery)
- Facilità di implementazione (quick win vs trasformazione profonda)
- Effetto moltiplicatore (un intervento che sblocca altri)

Esempio tipico di prioritizzazione:

- Priorità 1: introdurre 1-on-1 strutturate (impatto alto su team health e leadership, facile da implementare)
- Priorità 2: definire e comunicare chiari criteri di avanzamento di carriera (impatto su retention e motivazione)
- Priorità 3: ridurre WIP e introdurre definition of done (impatto su delivery effectiveness)

Struttura del report

Il report ha due livelli:

Executive summary (1 pagina — per CEO/CTO):

- Engineering Maturity Score per dimensione (grafico radar)
- Top 3 rischi operativi
- Top 3 raccomandazioni con stima dell'impatto
- Prossimi passi suggeriti

Report operativo (10–15 pagine — per Engineering Manager):

- Metodologia usata
- Risultati per dimensione con evidenze
- Interventi raccomandati con roadmap di implementazione
- KPI di successo per ogni intervento
- Template e strumenti pronti all'uso

Come consegnare il report

Non mandare il PDF via email e sparire. Sessione di debrief con CEO/CTO prima (executive summary, 45 minuti). Poi sessione con Engineering Manager (report operativo completo, 90 minuti). Questo crea ingaggio e permette di raccogliere obiezioni prima che diventino resistenze silenti.

SEZIONE 4 — Il Leadership Program (3 mesi)

4.1 Struttura generale

Il programma è basato sul presupposto che i comportamenti cambiano attraverso pratica ripetuta e feedback nel contesto reale — non attraverso formazione in aula.

Ogni mese ha un focus, dei milestone misurabili e delle sessioni di lavoro. Il ritmo standard:

- Sessione settimanale con Engineering Manager: 60–90 minuti
- Review mensile con CTO: 30 minuti (per allineamento)
- Osservazione diretta di una cerimonia al mese (standup, retro, 1-on-1 osservata)

4.2 Mese 1 — Foundation

Obiettivo: creare una base stabile di comprensione e definire dove vogliamo arrivare.

Assessment completato nella prima settimana se non già fatto. Il programma parte sempre dall'Assessment.

Definizione degli obiettivi: insieme all'Engineering Manager e al CTO, definire 3 KPI di successo del programma. Devono essere:

- Misurabili (numero, percentuale, frequenza)
- Verificabili a distanza di 3 mesi
- Significativi per il business, non solo per me

Esempi concreti:

- "Team satisfaction score passa da 3.1 a 4.0 (survey anonima)"
- "Deployment frequency passa da 1/2 settimane a 2/settimana"
- "100% del team ha 1-on-1 settimanale con l'EM entro il mese 2"

Primo quick win: ogni programma deve avere un risultato visibile entro le prime 3 settimane. Questo costruisce credibilità e motivazione. I quick win più efficaci in questo contesto:

- Introduzione delle 1-on-1 regolari (se non esistono)
- Prima retrospective strutturata con il team
- Definizione scritta dei criteri di code review

Le 5 conversazioni che ogni EM deve saper fare

Questo è il nucleo formativo del mese 1. Non sono teorie — sono competenze pratiche.

- La 1-on-1 efficace: non un aggiornamento di status. Una conversazione sulla persona, sulla sua crescita, sui suoi blocchi. Come tenerne una (template in Sezione 7).
- Il feedback critico: dare un feedback che fa male senza distruggere la relazione. Struttura SBI (Situation-Behavior-Impact). Come prepararsi prima, come gestire la difensività.
- La conversazione difficile: affrontare un problema comportamentale o di performance senza evitarlo. Come entrare nella conversazione, come uscirne con un accordo concreto.
- La delega reale: delegare non è "dai, fallo tu". È trasferire ownership, definire il livello di autonomia atteso, essere disponibile senza fare micromanagement.
- La comunicazione delle decisioni impopolari: come comunicare una decisione che il team non vorrà sentire (taglio di risorse, cambio di priorità, ristrutturazione). Come farlo in modo onesto e rispettoso.

Per ogni competenza: spiegazione teorica (15 min) + simulazione con roleplay (30 min) + debriefing (15 min). Il roleplay è fondamentale — la comprensione intellettuale non produce competenza comportamentale.

4.3 Mese 2 — Development

Obiettivo: lavorare sulle aree critiche emerse dall'assessment. Costruire nuovi pattern di comportamento.

Sessioni di coaching mensili con Engineering Manager: 2 ore al mese, strutturate in:

- Revisione di cosa è successo dall'ultima sessione (30 min)
- Lavoro su un tema specifico emerso (60 min)
- Piano d'azione per le prossime 2 settimane (30 min)

Il tema specifico dipende dall'assessment. I temi più frequenti:

- Come dare feedback critico a un senior developer
- Come gestire un conflitto tra due membri del team
- Come supportare un developer in difficoltà senza fare il suo lavoro
- Come comunicare verso l'alto (al CTO) quando il team è in difficoltà

Introdurre la feedback culture — approccio concreto

Non funziona organizzare un "corso sul feedback". Le persone ascoltano, annuiscono, e tornano a fare esattamente quello che facevano prima.

Quello che funziona:

Fase 1 — modelling: l'Engineering Manager comincia a chiedere feedback su se stesso in modo esplicito. Nelle riunioni di team: "Voglio migliorare come conduco le retrospective. Cosa posso fare diversamente?" Questo segnala che il feedback è sicuro.

Fase 2 — struttura: introdurre un formato per il feedback (SBI) e usarlo prima in contesti a basso rischio (code review, post-mortem tecnico). Non in conversazioni di performance.

Fase 3 — normalizzazione: il feedback diventa parte delle cerimonie esistenti. Ogni sprint retrospective ha uno slot "feedback al team e al manager". La frequenza crea abitudine.

Fase 4 — espansione: il feedback circola in tutte le direzioni — non solo dall'alto verso il basso. I developer si danno feedback tra loro, al Tech Lead, all'EM.

Questo processo richiede 6–8 settimane per stabilizzarsi. Non forzarlo.

Costruire la psychological safety — le 4 fasi di Edmondson

Amy Edmondson ha identificato 4 condizioni che creano psychological safety. Non le leggo — le pratico e le insegno come comportamenti concreti.

- Framing del lavoro come incertezza, non come esecuzione: il manager dice esplicitamente "non sappiamo come fare questa cosa — dobbiamo scoprirla insieme". Questo abbassa la barriera della paura di sbagliare.
- Modellare la vulnerabilità: il manager ammette errori suoi. "Ho sbagliato a stimare questo progetto. Ecco cosa avrei dovuto fare diversamente." Il team capisce che ammettere errori è sicuro.
- Risposta produttiva ai fallimenti: quando qualcuno sbaglia, la risposta del manager non è punizione o silenziosa disapprovazione — è curiosità. "Cosa ha portato a questa situazione? Cosa impariamo?"
- Inclusione proattiva: nelle riunioni, il manager chiede attivamente l'opinione delle persone più silenziose. "Mario, non ti ho ancora sentito su questo — cosa ne pensi?" Questo segnala che ogni voce conta.

La mia responsabilità nel programma è aiutare l'EM a praticare questi comportamenti nelle situazioni reali — non solo a capirli concettualmente.

4.4 Mese 3 — Stabilizzazione

Obiettivo: consolidare i nuovi comportamenti, rendere il team autonomo, misurare i risultati.

Consolidamento: nelle ultime 4 settimane, lavoro con l'EM su come mantenere i nuovi comportamenti senza dipendere da me. Questo significa:

- Routines consolidate (1-on-1, retrospective, feedback) che girano da soli
- Dashboard delle metriche che l'EM monitora in autonomia
- Una lista di "segnali di allarme" che indicano quando il sistema sta scivolando indietro

Handover: consegna all'EM e al CTO:

- Template e strumenti usati durante il programma
- Dashboard metriche con benchmark di riferimento

- Piano di azione per i prossimi 3 mesi (auto-gestito)

Post-assessment: ripeto la survey anonima del team (stessa del pre-assessment). Calcolo il delta sui KPI definiti nel mese 1. Questo è il momento della verità — e anche il materiale per il case study (anonimizzato) da usare come social proof.

Come presentare i risultati: sessione finale con CEO/CTO e EM. Formato: prima i risultati quantitativi (delta metriche), poi i risultati qualitativi (cambiamenti comportamentali osservati), poi le raccomandazioni per i prossimi 6 mesi. Questa sessione decide se il cliente rinnova o meno.

SEZIONE 5 — Strumenti per le Situazioni Difficili

5.1 Il developer senior tossico

Definizione: bravissimo tecnicamente, impatto devastante sul team. Comportamenti tipici: svaluta gli altri in code review, blocca le discussioni con "l'ho sempre fatto così", prende le decisioni unilateralmente, crea dipendenza da sé stesso per il codice critico.

Capire prima se è recuperabile

Non tutti i senior tossici lo sono per scelta. Alcune domande da fare all'EM prima di decidere l'approccio:

- Ha avuto feedback esplicito su questi comportamenti in passato? Come ha reagito?
- Il suo comportamento tossico è costante o è peggiorato in un periodo specifico? (Se peggiorato di recente, cercare cause: stress personale, frustrazione per una promozione mancata, burn-out)
- Ha relazioni funzionali con qualcuno nel team? O è universalmente difficile?

Se è recuperabile: serve una conversazione diretta, strutturata, senza ambiguità.

L'EM deve essere preparato a questa conversazione — lo simulo in roleplay con lui prima. La struttura:

- Aprire con l'impatto osservato, non con l'intenzione attribuita: "Nelle ultime 3 code review ho osservato che il tuo feedback usa un tono che mette le persone sulla difensiva. Ti do tre esempi specifici." (avere i tre esempi pronti)
- Chiedere la prospettiva: "Come vivi tu il lavoro con il team? Come pensi che gli altri ti percepiscano?"
- Definire l'aspettativa esplicita: "Quello che mi aspetto da te è [comportamento concreto]. Entro [data]."
- Definire le conseguenze: "Se non vediamo un cambiamento, il prossimo passo è [conseguenza]. Non te lo dico per minacciarti — te lo dico perché voglio che tu abbia le informazioni complete."
- Offrire supporto: "Cosa posso fare io per supportarti in questo cambiamento?"

Applicazione NLP — riformulazione: il senior tossico spesso non si percepisce come problema. Usa una riformulazione che non neghi la sua competenza ma sposti il frame: "Sei tecnicamente eccellente — il team lo riconosce. Il problema non è la tua competenza, è che le persone hanno smesso di imparare da te perché interagire con te è costoso. Questo è un problema per il team, ma anche per te: stai perdendo influenza."

Se non è recuperabile: il processo è delicato perché solitamente questa persona ha conoscenza critica nel codice.

Step prima di agire:

- Documentare i comportamenti problematici con date ed esempi (necessario per il processo HR)
- Fare un knowledge audit: cosa sa solo lui? Dove è bottleneck?
- Knowledge transfer sprint: prima di avviare il processo di uscita, pianificare 4–6 settimane di trasferimento della conoscenza (pair programming forzato, documentazione obbligatoria)

Step durante il processo:

- Coinvolgere HR e legal dall'inizio, non alla fine
- Comunicare al team in modo onesto ma rispettoso ("abbiamo preso strade diverse")
- Non lasciare un vuoto — avere già un piano per coprire le sue responsabilità

Attenzione al danno da uscita: un senior che esce con rabbia può fare danni (accesso ai sistemi, conversazioni negative con il team, cliente, recruiter). Gestire l'offboarding con attenzione.

5.2 Il tech lead fantasma

Definizione: nominato formalmente come leader tecnico, ma non guida nessuno. Non prendono decisioni, non danno direzione, non risolvono conflitti. Il team non li percepisce come reference point.

Assessment delle cause — tre possibilità con approcci diversi:

- Volontà assente: il Tech Lead non vuole essere un leader. È stato promosso per motivi "politici" o perché era il più senior. Non ha mai chiesto il ruolo.
- Approccio: conversazione esplicita sull'allineamento di aspettative. "Vuoi davvero fare questo ruolo?" Se la risposta è no, meglio saperlo ora.
- Competenza assente: vuole fare il leader ma non sa come. Non ha mai avuto modelli di riferimento, non ha ricevuto formazione.
- Approccio: programma di sviluppo su misura. Iniziare dalle competenze base (come fare una code review costruttiva, come facilitare una decisione tecnica, come gestire un disaccordo).
- Supporto assente: vuole fare il leader, ha le competenze di base, ma l'Engineering Manager non gli lascia spazio. L'EM decide tutto, bypassa il Tech Lead, lo tratta ancora come senior developer.
- Approccio: lavorare con l'EM sulla delega reale. Il Tech Lead non può emergere se l'EM non gli cede territorio.

Piano di sviluppo per caso 2 (competenza assente):

Mese 1 — definire il perimetro: cosa ci aspettiamo che il Tech Lead faccia in modo autonomo? (Esempio: guida le code review, facilita le decisioni tecniche nel dominio X, è il primo punto di contatto per i developer junior)

Mese 2 — pratica guidata: il Tech Lead prova i nuovi comportamenti con il mio supporto. Faccio da shadow (osservo e debriefing dopo), poi inversione (lui osserva me, poi prova da solo).

Mese 3 — autonomia: il Tech Lead gestisce in autonomia. Sessioni bisettimanali di check-in per supporto, non per supervisione.

5.3 La perdita di persone chiave

Il problema reale: molte organizzazioni tech hanno conoscenza critica concentrata su 1–2 persone. Quando escono (volontariamente o no), il team è in crisi.

Come identificare il rischio prima che sia tardi:

Segnali da monitorare: la persona smette di partecipare attivamente nelle cerimonie. Qualità del lavoro in calo. Richiesta di congedi improvvisa. LinkedIn aggiornato. Conversazioni più brevi e formali nelle 1-on-1.

Se uno di questi segnali è presente, non aspettare — fare una 1-on-1 diretta. "Ho notato che ultimamente sembri meno ingaggiato. Come stai? C'è qualcosa su cui posso supportarti?"

Knowledge documentation sprint

Se il rischio di uscita è concreto (o anche solo come pratica preventiva), pianificare uno sprint di trasferimento della conoscenza:

- Inventario delle aree di conoscenza critica (quali sistemi, processi, decisioni passate conosce solo questa persona)
- Sessioni di pair programming con almeno 2 altre persone per ogni area critica
- Documentazione scritte delle scelte architetturali chiave (ADR)
- Sessioni di Q&A aperte con il team su aree di competenza specifica

Non farlo sembrare un "processo di offboarding anticipato" — farlo sembrare (e farlo essere) un investimento nella crescita del team.

Dopo l'uscita

Non ignorare l'impatto emotivo sul team. La perdita di una persona chiave crea:

- Preoccupazione per il carico di lavoro aggiuntivo
- Incertezza sulla stabilità del team
- Possibile effetto domino (altri iniziano a cercare)

Gestire il momento con trasparenza: comunicare cosa cambia, chi copre cosa, quali sono i piani per compensare la mancanza. Non fare finta che non sia successo niente.

5.4 Come fare un hiring tecnico serio

Il processo in 4 fasi

Fase 1 — CV screening (10 minuti max)

Non cerco il CV perfetto. Cerco segnali di pensiero. Cosa guardo:

- Progressione di carriera: è cresciuta nel tempo? Ci sono salti logici?
- Impatto descritto: "ho implementato X" vs "ho contribuito a ridurre il tempo di build del 40%"
- Red flag: job hopping sistematico (ogni 6–8 mesi) senza motivo apparente, CV con solo buzzword senza contesto

10 minuti è realmente 10 minuti. Se dopo 10 minuti non ho trovato niente di interessante, no.

Fase 2 — Call di 30 minuti

Obiettivo: capire se vale la pena andare avanti. Non è un'intervista tecnica — è una conversazione.

Domande standard:

- "Parlami del progetto tecnico di cui sei più soddisfatto. Cosa hai fatto tu, non il team?"
- "Descrivimi una situazione in cui hai dovuto prendere una decisione tecnica difficile. Quali erano le opzioni?"
- "Cosa non ti piaceva del tuo ultimo lavoro? Cosa hai fatto per cambiarlo?"

La terza domanda è quella più informativa. Se la risposta è solo lamentela senza azione, è un segnale.

Fase 3 — Technical conversation (60–90 minuti, no whiteboard)

Non faccio coding su whiteboard. È uno stress test artificiale che misura la resistenza allo stress, non la competenza tecnica.

Faccio una conversazione tecnica su problemi reali che abbiamo affrontato (anonymizzati). Esempio: "Abbiamo un sistema con un'alta frequenza di aggiornamenti di stato che deve essere consistente tra più client. Come ti ci

avvicineresti?"

Quello che cerco:

- Come struttura il pensiero: chiede clarifying questions o parte a sparare soluzioni?
- Come gestisce l'incertezza: ammette di non sapere qualcosa e propone come scoprirlo?
- Profondità vs ampiezza: sa andare in profondità su un argomento o rimane in superficie su tutto?

Fase 4 — Pair session (2–3 ore con un membro del team)

Il candidato lavora su un problema reale (piccolo, bounded) con un membro senior del team. Obiettivo: osservare come interagisce nella pratica, come chiede aiuto, come gestisce il feedback immediato.

Debriefing obbligatorio con il membro del team dopo: "Lavoreresti bene con questa persona? Perché?"

Seniority reale — come valutarla

Non chiedo "sai cos'è il pattern strategy?". La risposta a questa domanda mi dice se il candidato ha letto un libro, non se sa applicarlo.

Domande per valutare seniority reale:

- "Hai mai introdotto un'astrazione che si è rivelata sbagliata? Cosa è successo?"
- "Quando hai spinto per una scelta tecnica che il team non condivideva, come l'hai gestita?"
- "Descrivi il peggior incident che hai gestito. Cosa hai imparato?"

Un senior developer risponde a queste domande con esempi concreti e ammette errori senza deflection. Un junior o mid risponde in astratto o non ricorda esempi specifici.

Red flag durante le interview (cosa evitare come intervistatore)

- Domande trick o puzzle ("quante palline ci stanno in un bus?"): misurano l'ansia, non la competenza
- Monopolizzare la conversazione: se parli più tu del candidato, non stai intervistando
- Giudizi prematuri nei primi 5 minuti: l'effect alone distorce tutto quello che viene dopo
- Non dare feedback al candidato al termine: è irrespettoso e danneggia il brand dell'azienda

SEZIONE 6 — NLP Applicato al Team Tech

6.1 Nota metodologica

Non uso il NLP come show — non voglio che le persone capiscano che sto usando tecniche. Lo uso per migliorare la qualità delle conversazioni, creare condizioni di sicurezza psicologica e aiutare le persone a sbloccarsi. In un contesto tech, il NLP funziona meglio quando è invisibile.

6.2 Rapport — entrare in sintonia con developer introversi

I developer tech hanno spesso una relazione con la comunicazione diversa dalla norma. Molti sono introversi, abituati a comunicare in forma scritta, diffidenti verso chi parla troppo e dice troppo poco.

Cosa funziona per creare rapport in questo contesto:

Mirroring del registro linguistico: se usano termini tecnici, li uso anch'io (senza fingere di non capirli). Se usano linguaggio preciso e asciutto, faccio lo stesso. Non parlare con tono da "coach motivazionale" — è il modo più rapido

per perdere credibilità.

Ritmo della conversazione: i developer spesso hanno pause più lunghe prima di rispondere. Non riempire quelle pause. Il silenzio non è imbarazzo — è pensiero. Aspettare. Chi ha fretta di riempire il silenzio perde informazioni preziose.

Evidenza di ascolto reale: riformulare quello che hanno detto con parole diverse. "Quindi quello che mi stai dicendo è che il problema principale non è il tool, ma come vengono prese le decisioni su quando usarlo. Ho capito bene?" Questo segnala che stai ascoltando, non solo aspettando il tuo turno di parlare.

Rispetto della distanza professionale: non forzare la dimensione personale troppo presto. Iniziare dal lavoro, dalla tecnica, dal concreto. La dimensione personale emerge da sola quando il rapport è solido.

6.3 Calibrazione in video call

La video call toglie il 60% dei segnali non verbali (postura, movimento, prossimità). Quello che rimane:

Voce: tono, ritmo, volume, pause. Una voce che si abbassa e rallenta su un argomento specifico indica tensione o emozione. Una risposta troppo rapida e fluente su una domanda difficile può indicare risposta difensiva preparata.

Viso: micro-espressioni visibili (sorriso non raggiunge gli occhi, tensione mascellare, sopracciglia alzate di fretta). Non fare diagnosi — usarle come segnali di attenzione su cui tornare.

Contatto visivo vs sguardo laterale: in video call, le persone che guardano in basso o lateralmente quando parlano di certi argomenti spesso stanno accedendo a emozioni o ricordi. Non è menzogna — è elaborazione.

Cosa fare con queste informazioni: non commentarle esplicitamente. Usarle per sapere su quali argomenti tornare e per regolare il tono della domanda successiva. Se la persona sembra a disagio su un tema, usare una domanda più morbida, non più diretta.

6.4 Riformulazione — dare feedback critico senza triggerare difensività

La difensività è una risposta automatica del sistema nervoso a quello che percepisce come attacco. Non è irrazionalità — è protezione. Il mio compito non è sopprimerla, ma non attivarla inutilmente.

Tecniche concrete:

Separare comportamento da identità: "Questa code review è stata poco utile" è un giudizio sul comportamento. "Sei un cattivo code reviewer" è un giudizio sull'identità. Il secondo attiva difensività molto più del primo.

Descrivere l'impatto prima del giudizio: invece di "hai sbagliato", dire "quando hai fatto X, l'effetto sul team è stato Y. Voglio capire cosa ti ha portato a farlo."

Domanda invece di affermazione critica: invece di "non gestisci bene le priorità", chiedere "come hai deciso di priorizzare queste due cose? Voglio capire il tuo ragionamento." Spesso la risposta rivela che il problema non è dove pensavi.

Validare prima di correggere: trovare qualcosa di reale e positivo prima di arrivare al punto critico. Non è un sandwich di complimenti falsi — è riconoscere ciò che funziona prima di parlare di ciò che non funziona. Deve essere genuino.

6.5 Ancoraggio — associare stati positivi a comportamenti desiderati

L'ancoraggio è la capacità di creare associazioni tra uno stato emotivo positivo e un comportamento specifico. In pratica: quando qualcuno fa qualcosa di buono, il riconoscimento deve arrivare nel momento giusto per creare un'associazione duratura.

Come funziona in pratica nel team tech:

Dopo una buona code review: non aspettare la retrospective per riconoscerla. Il riconoscimento immediato, specifico e genuino ("ho visto la tua code review di ieri sul PR di Luca — hai spiegato il perché della tua obiezione in modo che lui ha capito cosa cambiare, non solo cosa era sbagliato") crea un'associazione positiva tra quel comportamento e il riconoscimento.

Dopo una buona retrospective: il facilitatore (EM o Tech Lead) chiude la sessione notando cosa ha funzionato bene nella conversazione stessa, non solo negli output. "Oggi ho visto persone dire cose difficili con rispetto — questo è esattamente il tipo di conversazione che voglio che facciamo regolarmente."

Nota: l'ancoraggio funziona solo se il riconoscimento è genuino e specifico. Il generico "ottimo lavoro" non ancora niente — è rumore di fondo.

6.6 Outcome Frame — focalizzare sui risultati invece che sui problemi

Il problem frame porta le persone a girare attorno al problema: "perché è successo questo, chi è responsabile, cosa non va." Utile per l'analisi, bloccante per l'azione.

L'outcome frame sposta il focus: "dove vogliamo arrivare, cosa vogliamo invece di questo problema, cosa farebbe la differenza."

Come usarlo nelle conversazioni di team:

Nelle retrospective: dopo aver identificato un problema, fare immediatamente la domanda di outcome: "Ok, questo è il problema. Come vorremmo che funzionasse invece? Cosa vorremmo vedere succedere diversamente nel prossimo sprint?"

Nelle 1-on-1: quando qualcuno porta un problema, usare "e cosa vorresti invece?" oppure "se questo fosse risolto, come sarebbe diverso il tuo lavoro?". Queste domande spostano dalla lamentela alla proposta.

Nelle conversazioni di performance difficili: invece di parlare di cosa non va, iniziare da dove vogliamo arrivare: "Parliamo di dove voglio che tu sia tra 3 mesi. [descrizione concreta]. Cosa ti separa da quel punto adesso?"

SEZIONE 7 — Template e Checklist

7.1 Survey Team Health (anonima, 15 domande)

Da mandare prima dell'Assessment e ripetere dopo il programma. Scala 1–5 (1 = fortemente in disaccordo, 5 = fortemente d'accordo). Strumento consigliato: Google Forms o Typeform.

BLOCCO A — Psychological Safety (4 domande)

- "In questo team mi sento al sicuro nell'ammettere un errore senza temere conseguenze negative."
 - "In questo team posso dire cosa penso davvero, anche se non è la cosa più comoda da sentire."
 - "In questo team le persone si rispettano anche quando non sono d'accordo."
 - "Se avessi un problema personale che influenza il mio lavoro, mi sentirei a mio agio nel dirlo al mio manager."
-

BLOCCO B — Leadership e Feedback (4 domande)

- "Il mio manager mi dà feedback utile e regolare su come sto lavorando."
 - "Quando mi viene dato un feedback critico, di solito è specifico e mi aiuta a capire cosa cambiare."
 - "Il mio manager mi supporta nella mia crescita professionale."
 - "Le decisioni importanti nel team vengono spiegate e hanno senso per me."
-

BLOCCO C — Delivery e Chiarezza (4 domande)

- "So sempre chiaramente cosa devo fare e perché è prioritario."
 - "Posso portare a termine il mio lavoro senza essere interrotto da blocchi evitabili."
 - "Il team consegna quello che si impegna a consegnare nella maggior parte degli sprint."
 - "C'è una quantità di lavoro sostenibile — non mi sento costantemente in emergenza."
-

BLOCCO D — Domanda aperta (1 domanda)

- "C'è qualcosa che cambieresti nel funzionamento del team se potessi? (risposta libera)"
-

Due domande di exit:

- "Quanto sei soddisfatto del tuo lavoro qui? (1 = per niente, 10 = moltissimo)"
 - "Quanto è probabile che tu stia ancora qui tra 12 mesi? (1 = molto improbabile, 5 = molto probabile)"
-

7.2 Template 1-on-1 Engineering Manager

Cadenza: settimanale, 30–45 minuti. Mai saltare. Se non c'è molto da dire, è una 1-on-1 di 15 minuti — ma si tiene.

Struttura:

1-on-1 — [Nome] — [Data]

CHECK-IN (5 min)

Come stai questa settimana? (scala 1-10 + una parola)

Cosa ha funzionato bene?

Cosa è stato difficile?

TEMI DELL'EM (15 min)

- Tema 1: [da preparare prima]

- Tema 2: [da preparare prima]

TEMI DEL COLLABORATORE (10 min)

(lasciare sempre spazio — la persona porta i propri temi)

FOLLOW-UP DALLA SETTIMANA PRECEDENTE

- Azione 1: fatto / non fatto — perché?

- Azione 2: fatto / non fatto — perché?

AZIONI PER LA PROSSIMA SETTIMANA

- [Chi fa cosa entro quando]

Note operative:

- Non usare la 1-on-1 come status update di progetto. Per quello ci sono i daily standup.
- Preparare i propri temi la sera prima. Se arrivi senza preparazione, il messaggio implicito è "tu non sei importante abbastanza da meritare i miei 10 minuti di preparazione".
- Tenere un documento condiviso con il collaboratore dove si tracciano le note e le azioni. Questo crea continuità e accountability bidirezionale.

7.3 Template 1-on-1 Developer (condotto dall'EM)

1-on-1 — [Nome] — [Data]

APERTURA

Come stai? Cosa ti ha dato più energia questa settimana?

LAVORO

C'è qualcosa che ti sta bloccando o rallentando?

C'è qualcosa su cui ti sentiresti più efficace con più supporto?

CRESCITA

Su cosa vorresti lavorare o imparare nel prossimo mese?

Come senti che stai crescendo?

TEAM E RELAZIONI

C'è qualcosa nel team che non funziona come vorresti?

Come vivi la collaborazione con [collega specifico se ci sono segnali]?

FEEDBACK SU DI ME (COME EM)

C'è qualcosa che potrei fare diversamente per supportarti meglio?

AZIONI

[Lista azioni con owner e deadline]

7.4 Template Retrospective (Start/Stop/Continue + Actions)

Durata: 60–90 minuti per team di 5–10 persone.

Facilitatore: Engineering Manager o Tech Lead (a rotazione dopo i primi 2 mesi).

Struttura:

RETROSPECTIVE — Sprint [N] — [Data]

Facilitatore: [Nome]

Partecipanti: [Lista]

ICEBREAKER (5 min)

Una parola che descrive questo sprint.

DATI (10 min)

Sprint goal: raggiunto / parzialmente / no

Velocity: [X] punti (target era [Y])

Incident e blocchi notevoli: [lista]

START (15 min)

Cosa dovremmo cominciare a fare che non stiamo facendo?

(Post-it individuali, poi votazione delle priorità)

STOP (15 min)

Cosa dovremmo smettere di fare?

(Post-it individuali, poi votazione delle priorità)

CONTINUE (10 min)

Cosa funziona bene e dobbiamo proteggere?

(Post-it individuali)

AZIONI (15 min)

Dalle priorità emerse, definire massimo 3 azioni.

Per ogni azione: chi la fa, entro quando, come sappiamo che è fatta.

Azione	Owner	Deadline	Done when
-----	-----	-----	-----
...	...	...	...

CHIUSURA (5 min)

Rating della retrospective stessa: 1-5

Cosa avremmo potuto fare meglio in questa retro?

Note operative:

- Massimo 3 azioni per retro. Più di 3 non vengono fatte. Meglio meno azioni eseguite che molte dimenticate.
- Le azioni devono avere owner e deadline — non "il team farà". Il team non fa niente, le persone fanno.
- Alla retro successiva, prima cosa: review delle azioni della retro precedente. Se non si fa follow-up, le persone smettono di credere alla retro.

7.5 Template Feedback Strutturato (SBI)

SBI = Situation — Behavior — Impact

Struttura:

SITUATION (contesto specifico)

"Nella code review di martedì scorso sul PR #247..."

BEHAVIOR (comportamento osservato, non interpretato)

"...hai scritto 'questo codice è orribile' senza spiegare cosa non andava..."

IMPACT (effetto concreto sul lavoro o sulle relazioni)

"...Marco mi ha detto che si è sentito svalutato e ha impiegato il doppio del tempo a capire cosa cambiare, perché non aveva indicazioni chiare."

(OPZIONALE — RICHIESTA DI CAMBIAMENTO)

"Quello che mi aspetto in futuro è che il feedback in code review sia specifico: cosa non funziona e perché."

(OPZIONALE — ASCOLTO)

"Come la vedi tu?"

Note operative:

- Il feedback funziona solo se è tempestivo (entro 24–48h dall'evento) e privato (mai feedback critico in pubblico).
- La parte "impact" è la più importante — e spesso la più omessa. Senza impatto, il feedback è solo un'opinione.
- Finire con "come la vedi tu?" non è debolezza — è ascolto. A volte il comportamento ha una causa che non conoscevi.

7.6 Checklist Onboarding Nuovo Membro del Team

PRIMA SETTIMANA**Setup tecnico:**

- ☐ Accessi a tutti i sistemi (repository, CI/CD, monitoring, ticket)
- ☐ Ambiente di sviluppo funzionante (almeno un servizio che gira in locale)
- ☐ Spiegazione della pipeline CI/CD (come si deploia, come si fa rollback)
- ☐ Accesso agli strumenti di comunicazione e regole d'uso

Contesto team:

- ☐ Sessione di onboarding con EM (storia del team, cultura, aspettative)
- ☐ Presentazione 1-on-1 con ogni membro del team (15 min ciascuno)
- ☐ Overview dell'architettura tecnica (30–45 min con Tech Lead)
- ☐ Accesso e orientamento nel backlog (come lavorano i ticket, definition of done)

PRIMA SETTIMANA — Compito assegnato:

- ☐ Un ticket piccolo ma reale da portare in produzione entro la fine della settimana (obiettivo: prendere confidenza con il processo, non dimostrare la competenza tecnica)

PRIMO MESE

- ☐ Prima 1-on-1 con EM (entro il giorno 3)
- ☐ Sessione di pairing con ogni membro senior del team (almeno una)
- ☐ Partecipazione alla prima retrospective
- ☐ Feedback formale dall'EM a fine primo mese: cosa va bene, cosa sviluppare

PRIMO TRIMESTRE

- ☐ Check-in a 30 giorni: "come stai? cosa hai imparato? cosa ti manca ancora?"
- ☐ Check-in a 60 giorni: "dove senti di poter contribuire di più?"
- ☐ Check-in a 90 giorni: sessione strutturata di feedback bidirezionale
- ☐ Revisione degli obiettivi iniziali con l'EM

7.7 Template Engineering Manager OKR

OKR ENGINEERING MANAGER — Q[N] [Anno]

Nome: [Nome]

Review date: [Data]

OBJECTIVE 1: [Team ad alta performance e psicologicamente sicuro]

KR 1.1: Team Health Score (survey anonima) $\geq 4.0/5.0$ entro fine quarter

KR 1.2: 100% del team ha 1-on-1 settimanale con follow-up documentato

KR 1.3: Turnover involontario = 0 durante il quarter

OBJECTIVE 2: [Delivery prevedibile e di qualità]

KR 2.1: Sprint commitment accuracy $\geq 80\%$ (media delle ultime 6 settimane)

KR 2.2: Deployment frequency $\geq$ [target specifico] deploy/settimana

KR 2.3: Zero critical bug non rilevati in produzione per ≥ 30 giorni senza fix

OBJECTIVE 3: [Crescita del team]

KR 3.1: Ogni membro del team ha un piano di sviluppo scritto e aggiornato

KR 3.2: Almeno 2 sessioni di knowledge sharing interne condotte dal team

KR 3.3: Almeno 1 membro del team assume responsabilità nuove rispetto al quarter precedente

SELF-DEVELOPMENT (EM su se stesso)

[] Competenza da sviluppare: [es. gestione conversazioni difficili]

[] Azione concreta: [es. 3 roleplay con coach entro metà quarter]

[] Feedback da raccogliere: [es. da 3 persone del team sul mio stile di 1-on-1]

Note finali operative

Cosa non negoziare mai con il cliente:

- L'Engineering Manager deve partecipare attivamente. Se l'EM non c'è o non è ingaggiato, il programma non parte.
- Le sessioni di coaching sono confidenziali. Il CTO non può sapere cosa viene detto nelle sessioni individuali con l'EM — altrimenti la fiducia crolla.
- I KPI di successo vengono definiti prima, non dopo. Se il cliente non accetta di misurarsi, non accetta la responsabilità dei risultati.

Come mantenere la qualità nel tempo:

- Dopo ogni engagement, scrivere un post-mortem privato: cosa ho imparato, cosa avrei fatto diversamente, quale pattern si è ripetuto.
- Aggiornare questo playbook ogni 6 mesi con quello che ha funzionato e quello che no.
- Tenere un log dei feedback ricevuti dai clienti (anche informali) per identificare pattern di miglioramento.

Cosa tenere a mente nella mia testa quando lavoro:

Il mio valore non è sapere più del cliente. È la qualità delle domande che faccio e la capacità di stare in conversazioni difficili senza perdere il centro. La tecnica è al servizio della relazione, non il contrario.